# ArUco Camera Pose Estimation Study Guide

A practical learning path for camera geometry and robot frames

Prepared for Gajendra | IIT Patna | 5 November 2026
Reference status | Original upstream code pinned at 236c420d3470 | MIT license
Original repository

## Purpose and status

This guide explains a compact ROS 2 project that estimates a camera pose from a known ArUco marker. The goal is to learn camera intrinsics, marker detection, coordinate transforms, service calls and validation. The upstream repository belongs to Davide Nardi. It is included as a pinned reference submodule. The package was reviewed from source but not built or run with a camera during preparation of this guide.

## Abstract

The upstream node subscribes to a camera image topic, detects a requested marker ID, estimates the marker pose in the camera frame, collects several observations, and returns a camera pose through a ROS 2 service. The learner should first trace frames and units, then supply real camera calibration values and measure stability. The proposed independent extension adds quality checks and a clear diagnostic report. These skills matter whenever a robot must relate image measurements to physical workspace coordinates.

## Why this reference is useful

The project is small enough to read its C++ image processor and service callback in one sitting. It exposes the practical difference between intrinsic camera calibration and extrinsic camera pose. It also shows a service-based workflow rather than a continuous output topic. The repository has a small public footprint and includes build workflows for Humble and Jazzy; its runtime behavior still needs independent validation on the learner's hardware.

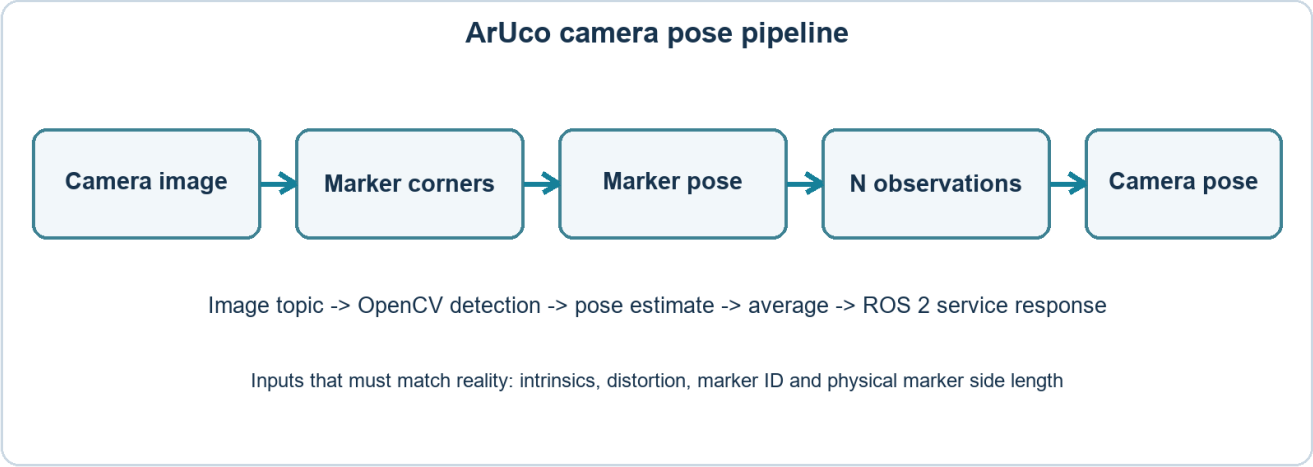



*Figure 1 Data path inferred from the pinned upstream image processor and service code*

## The geometry in plain language

Camera intrinsics describe how a 3D point maps to pixels. The focal lengths fx and fy and principal point cx and cy form a 3 by 3 camera matrix. Distortion coefficients describe how the lens bends straight image rays. Extrinsics describe where the camera sits relative to a marker, robot base or world frame. Correct intrinsics and a measured marker side length are prerequisites for a meaningful metric pose.

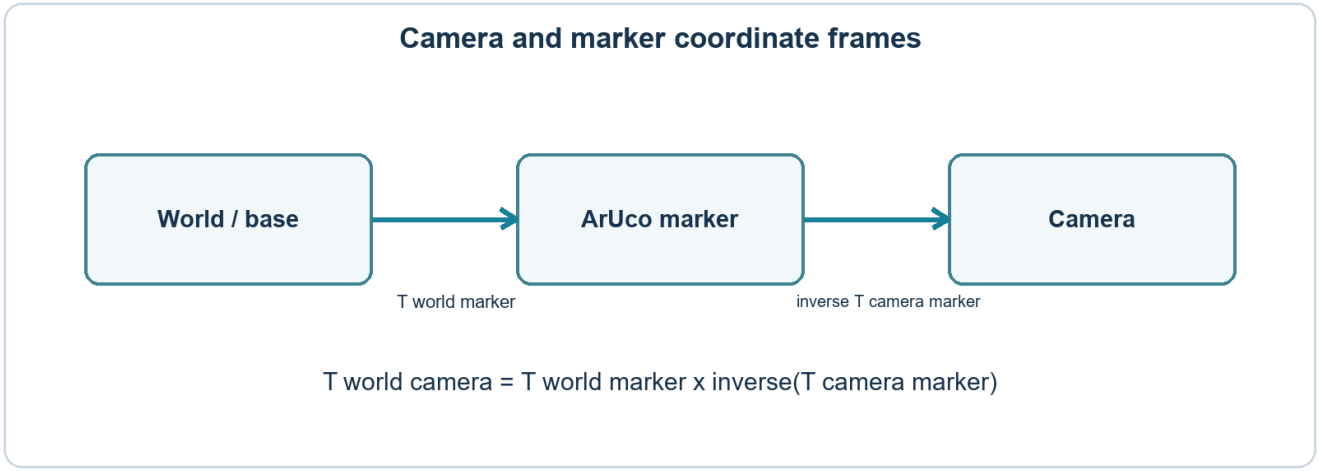



*Figure 2 Coordinate chain used to express camera pose in a chosen world or base frame*

The marker detector estimates a transform from marker coordinates into camera coordinates. To express the camera in a world frame, invert that detected transform and combine it with the known world-to-marker transform. In compact notation: T_world_camera = T_world_marker x inverse(T_camera_marker). A transform is a 4 by 4 matrix carrying rotation and translation. Always state the source and target frames before multiplying transforms.

## Read the upstream files in this order

| File | Question to answer |
|---|---|
| README.md | What does the service return and what hardware is assumed? |
| calibration_srv/srv/Calibration.srv | What enters the service and what pose message comes back? |
| aruco_camera_pose_estimator_cpp/config/config.yaml | What image topic, marker ID context, sample count, timeout and frame are configured? |
| aruco_camera_pose_estimator_cpp/config/calibration_params.yaml | Which intrinsics and distortion values are example values? |
| aruco_camera_pose_estimator_cpp/src/image_processor.cpp | How are markers matched, pose vectors computed and observations averaged? |
| aruco_camera_pose_estimator_cpp/src/pose_service.cpp | How is the service called, timed out and transformed into the response frame? |

## Parameters that must be checked

| Parameter | Why it matters | Action before a real run |
|---|---|---|
| input_topic_name | Names the incoming image stream | Confirm with ros2 topic list and echo |
| aruco_size | Sets metric scale of translation | Measure printed marker side and use metres as the code/config expect |
| fx fy cx cy | Camera matrix at chosen image resolution | Calibrate your own camera or load verified values |
| k1 k2 k3 p1 p2 | Lens distortion model | Use measured coefficients, not copied defaults |
| n_observation | Number of detections averaged | Compare latency and pose variation |
| timeout_ms | Maximum wait for sufficient detections | Check actual camera frame rate and visibility |
| frame_id and marker offset | Defines output coordinate frame | Measure marker pose relative to the chosen base |

At the pinned commit, the example calibration file contains values for an RS435 at 1280 by 720 and zero distortion coefficients. Treat those as sample values. The README describes marker size in centimetres, while config.yaml and pose_service.cpp label aruco_size as metres. Follow the code's unit convention and verify scale with a known distance before trusting a pose.

## Environment and first run

Use a supported Ubuntu and ROS 2 environment with a camera driver that publishes a ROS image topic. The upstream README lists OpenCV development files and cv_bridge, then builds the C++ package and custom service with colcon. A printed DICT_4X4_50 marker of measured side length is needed. The repository does not supply a camera driver or intrinsics for every webcam. Windows WSL2 can build ROS code, but passing a physical camera into the Linux environment may require extra setup; native Ubuntu is often simpler for the first camera run.

```
# In a sourced Ubuntu ROS 2 shell
mkdir -p ~/ros2_ws/src && cd ~/ros2_ws/src
git clone https://github.com/Hydran00/aruco_camera_pose_estimator.git
cd ..
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
ros2 launch aruco_camera_pose_estimator_cpp aruco_camera_pose_estimator.launch.py
# In another shell, call /calibration/get_camera_pose with marker_id
```

These are study instructions and were not executed here. Before service calls, check that the image topic is live, marker ID and dictionary match the printed marker, the camera intrinsics match the stream resolution, and the marker has a measured world or base pose. Capture the exact service response and verify its header frame and units.

## Validation experiments and evidence

| Experiment | Action | Evidence to save |
|---|---|---|
| Single marker | Hold one marker at a measured distance | Image, marker size, pose, distance error |
| Repeated calls | Request pose at a fixed camera position | Translation and rotation variation across runs |
| Different views | Tilt and move the marker within the image | Pose changes and detection failures |
| Wrong configuration | Change marker ID or image topic deliberately | Timeout and error response |
| Two markers | Show requested and unrelated IDs together | Which marker drives the response |
| Resolution change | Switch image size without recalibration | Scale or bias change; then correct intrinsics |

Use a table with date, camera, resolution, calibrated matrix, distortion values, marker ID, measured side length, number of detections, output frame, measured reference distance and computed error. Save sample images only when allowed. Do not invent accuracy numbers from a single screenshot.

## Code review question for multiple markers

At the pinned commit, image_processor.cpp loops over detected marker IDs, but after calling estimatePoseSingleMarkers for the full corners list it reads tvec[0] and rvec[0]. Inspect whether the selected ID can be at another index and design a test with two visible markers. This is a testable review finding, not a confirmed field failure. A later independent patch should choose the pose matching the requested marker and include a regression test.

## Original follow-on project brief

Build a separate Camera Pose Quality Monitor after reproducing the upstream baseline. The new tool can accept frames and a calibrated camera model, select a specified marker ID, estimate pose, calculate corner reprojection error, reject low-quality frames, and export a CSV with timestamps and pose variation. Give it a small visualization that marks detected corners and axes. Keep the upstream project credited and implement your own tested code in a new repository.

| Deliverable | Minimum acceptance check |
|---|---|
| Marker selection | Requested ID is used when multiple IDs are visible |
| Unit verification | Known marker side and distance yield a documented scale error |
| Quality metric | Every accepted pose has a reprojection error and threshold |
| Repeatability | Report mean and spread of pose over a stationary capture |
| Failure cases | No marker, blurred marker, wrong calibration and timeout are logged |

Start with a recorded image set or a webcam before integrating a robot arm. The goal is a reproducible calibration check, not a claim of universal accuracy. Later, connect the pose to a robot frame tree and test a pick-point mapping with a measured target.

## Four week learning plan

| Week | Output you should personally produce |
|---|---|
| 1 | Frame diagram, source-code map and camera calibration plan |
| 2 | Working image topic, marker detection and first service response |
| 3 | Repeated pose measurements, multi-marker test and error analysis |
| 4 | Independent quality checker with tests, plots and a measured report |

## Interview questions to practise

- What is the difference between camera intrinsics and camera extrinsics?
- Why does a wrong printed marker size distort translation scale?
- Which transform must be inverted to express camera pose in the world frame?
- How would you know whether an estimated pose is trustworthy?
- Why might a service request time out even when a camera topic is visible?

## What can honestly appear on a resume

After a verified run, state the camera and calibration process, frame convention, measured repeatability or error, and code or tests you personally wrote. Until then, label this as a studied upstream reference. A strong original entry links to your own dataset, validation plots and implementation changes.

## Sources and attribution

Pinned upstream project https://github.com/Hydran00/aruco_camera_pose_estimator

OpenCV ArUco detection https://docs.opencv.org/5.0/tutorials/objdetect/aruco_detection/aruco_detection.html

OpenCV camera calibration
https://docs.opencv.org/5.0/tutorials/calib3d/camera_calibration/camera_calibration.html

ROS 2 Jazzy documentation https://docs.ros.org/en/jazzy/